

CodeZen Interview Cheat Sheet

20 High-Quality Questions & Short Answers

Prepared for: Shakti Kumar Sah • KIIT University

AI Tutor Integration (Gemini API)

Q1 How did you integrate Gemini and force it to act as a tutor?

I integrated the Gemini API on the backend. By writing a detailed **System Prompt** that wraps the user's code and execution errors, I forced the model to only output **hints, dry-run checklists, and complexity advice**, explicitly forbidding it from printing the full solution.

Q2 What happens if the Gemini API goes offline or fails?

I built an **offline fallback mechanism**. If the API fails, the backend locally parses the compiler logs and outputs a static, rule-based debugging checklist depending on whether it's a Compilation Error, Runtime Error, or Wrong Answer.

Q3 What parameters does the backend send to the Gemini API controller?

The frontend payload sends the user's active code, the current language, the problem title, the problem description, previous conversation history (for continuity), and the compiler's exit/run status.

Q4 How does the AI analyze space and time complexity?

The system prompt instructs Gemini to review loop structures and variables in the user's code, estimate the worst-case complexities in Big O notation, and prompt the candidate to optimize nested loops or high memory allocations.

Code Evaluation & Sandboxing

Q5 How do you execute user code safely?

To prevent security breaches (like system access or infinite loops), we never run code locally. Instead, we offload execution to a **remote compiler sandbox** (OneCompiler via RapidAPI) which executes the code in isolation and returns output logs safely.

Q6 How are inputs parsed and fed into the user's functions?

Before sending the code, the backend automatically wraps it with **language-specific runner templates**. It parses the inputs from MongoDB and dynamically writes driver code (like `int main()` in C++ or `main` classes in Java) to execute the target function.

Q7 How did you bypass RapidAPI's 1-request-per-second limit?

I implemented an **asynchronous sequential delay queue**. When executing a batch of test cases, the system introduces a 1.1-second sleep delay between each test case request, ensuring we never trigger rate limits.

Q8 How are the user's execution run logs compared with target test cases?

We capture stdout from the remote execution result, strip trailing whitespace and carriage returns, and perform a strict string comparison with the expected outputs stored in MongoDB.

Authentication & Session Security (Redis)

Q9 Why did you use Redis for token blacklisting?

JWT tokens are stateless and cannot be revoked by the server once issued. To support secure logouts, we store logged-out token signatures in **Redis**, blocking any hijacked requests using that token.

Q10 How do you optimize Redis memory usage for blacklisted tokens?

Every blacklisted token is saved with a **Time-to-Live (TTL)** parameter matching the exact remaining expiration time of the JWT. Once the token naturally expires, Redis automatically purges it, keeping RAM usage extremely low.

Q11 How does the JWT verification middleware work?

It intercepts incoming requests, extracts the token from the Authorization header, verifies its integrity using our secret key, and queries Redis. If the token key exists in Redis, the request is rejected with a 401 Unauthorized status.

Q12 How do you protect MongoDB queries against NoSQL injection?

By utilizing the **Mongoose ODM**. Mongoose enforces strict schema definitions and sanitizes query input automatically, casting values to match schema types (such as casting strings to ObjectIDs), preventing query injection.

 Database & State Management**Q13 Why choose MongoDB instead of a SQL database like MySQL?**

Coding challenges require flexible schema structures to handle multiple language templates, variables, and arrays of test cases. MongoDB's **document model** allows us to store all this nested data inside a single document without complex joins.

Q14 How did you handle CORS issues in local development?

I configured **Vite's reverse proxy** on the frontend to map all /api requests to the Express port (4000). This tricks the browser into believing all requests originate from the same port, bypassing CORS policies cleanly.

Q15 How did you display user statistics on the Profile page?

I used MongoDB aggregation pipelines to group all submissions by user ID and status. This allowed us to calculate totals, display accuracy rates, and render progress bars based on difficulty levels (Easy, Medium, Hard).

Q16 How does the Admin challenge form handle starter code?

It provides fields to enter initial code setups for multiple languages (JS, Python, Java, C++). These templates are stored inside a `startCode` subdocument array inside MongoDB, which is fetched dynamically when a user loads a problem.

Q17 What is Redux Toolkit and how did you use it?

Redux Toolkit is a high-level library for global state management. In our app, it stores auth status, user metadata, active filters, and loading states, ensuring consistency across the Workspace, Profile, and Admin Panel.

Q18 How does the sidebar filtering logic work on the backend?

The frontend appends query variables (difficulty, tags, search queries) to the API request. The Express controller extracts these query parameters and builds a dynamic MongoDB query object using operators like `$regex` and exact matches.



Scalability & Deployment

Q19 How did you deploy the frontend and backend?

The frontend is deployed on **Vercel** for fast global delivery and automatic builds. The backend API is hosted on a cloud provider with MongoDB Atlas as the managed cloud database.

Q20 If the system scales to millions of users, how would you redesign the code runner?

I would introduce a **Message Queue** (like RabbitMQ or Apache Kafka) between the Express backend and the runner. Submissions would be queued, and a pool of independent, auto-scaling worker nodes would pull from the queue to process them asynchronously.